

Exercițiu 1

-- Query A:

```
SELECT * FROM Orders WHERE customer_id = 42 AND status = 'pending';
```

-- Query B:

```
SELECT * FROM Orders WHERE status = 'pending' ORDER BY created_at;
```

-- Query C:

```
SELECT customer_id, SUM(total)
```

```
FROM Orders
```

```
WHERE created_at BETWEEN '2025-01-01' AND '2025-12-31'
```

```
GROUP BY customer_id;
```

- Adăugați 1 sau 2 indecși pentru fiecare interogare
- Justificați ordinea coloanelor
- Este suficient un singur index compus pentru toate interogările ? Justificați

Pentru query a

```
CREATE INDEX i_a ON ORDERS(customer_id, status)
```

Pentru query b

```
CREATE INDEX i_b ON ORDERS(status, created_at)
```

Pentru query c

CREATE INDEX i_c ON ORDERS(created_at, customer_id) include (total)

Ordinea se justifica prin ordinea prezentata la seminar, si anume

1. FROM + JOIN
2. WHERE
3. GROUP BY, HAVING
4. SELECT
5. ORDER BY

Nu este suficient cate un index compus pentru toate interogările deoarece crearea unui index compus, ca sa fie eficient trebuie sa respecte urmatoarea regula:

De exemplu, daca cream un index (a,b,c)

Atunci acesta poate fi util pentru

Where a =....

Sau Where a = .., b =....

Sau Where a = ..., b =..., C =....

Dar nu va fi util pentru un query de forma where b =..., c =..., sau care incepe cu c = ... si asa mai departe.

Avand in vedere ca indecsii corecti pentru aceste query-uri sunt cei prezentati mai sus, se poate observa ca niciunul nu incepe cu aceeasi coloana, motiv pt. care nu se poate face un index comun.

Exercițiu 2 – Index Killers

-- Tabela are un index pe coloana **name**. De ce nu este utilizat ?

1. **SELECT * FROM Orders WHERE EXTRACT (YEAR FROM created_at) = 2026 ;**

-- Tabela are un index pe coloana **name**. De ce nu este utilizat ?

2. **SELECT * FROM Customers WHERE LOWER(name)='popescu';**

-- Tabela are un index pe coloana **total**. De ce nu este utilizat ?

3. **SELECT * FROM Orders WHERE total > 500 OR status='pending';**

-- Similar, avem index pe coloana **email**, dar nu este folosit.

4. **SELECT * FROM Orders WHERE email LIKE '%yahoo.com';**

● Rescrieți interogările sau indecșii astfel încât indecșii să fie utilizați

1. Nu este utilizat deoarece condiția where din query se refera doar la coloana **created_at**, iar un index pe coloana **name** nu ajuta

REZOLVARE: refacem index-ul

CREATE INDEX i_year ON Orders (EXTRACT (YEAR FROM created_at))

2. Nu este utilizat deoarece index-ul creat pe coloana **name**, iar in query se cauta dupa coloana **name** dar convertita la litere mici.

Rezolvare: refacem index-ul pe coloana name dar convertita la lower

CREATE INDEX i_name_lowercase ON Customers(LOWER(name))

3. Indexul nu este utilizat deoarece predicatul “or” pe doua coloane diferite impiedica folosirea unui index eficient, arborele nu poate evalua doua coloane distincte deodata.

Rezolvare:

-- cream un index nou pe coloana **total** – pt primul query

Create index i_t on Orders(total)

■ Cream un index nou pe coloanele **status** si **total** – pt al doilea query

Create index i_tp on Orders(status, total)

Motivul pentru care este necesar acest index este faptul ca in al doilea query avem

egalitate pentru coloana **status**, si semn de interval pentru **total**. La crearea

indexurilor, coloanele pe care exista egalitate au prioritate fata de coloanele care au

semne de interval (<, > etc) si fata de coloanele care au inegalitate <>

Un singur index nu ar fi fost de ajuns pentru ca nu se respecta ordinea.

■ Rescriere query

SELECT * FROM ORDERS WHERE total > 500

UNION ALL

SELECT * FROM ORDERS WHERE total <= 500 AND status = 'pending'

-- Similar, avem index pe coloana **email**, dar nu este folosit.

4. **SELECT** * **FROM** Orders **WHERE** email LIKE '%yahoo.com';

-- Activează extensia pentru pattern matching

CREATE EXTENSION **IF** NOT **EXISTS** pg_trgm;

-- Index GIN pe email

CREATE INDEX idx_email_trgm **ON** orders **USING** GIN(email gin_trgm_ops);

Exercițiu 3

-- Interogarea următoare este executată de milioane de ori pe zi

SELECT status, total **FROM** Orders **WHERE** customer_id = 42;

- Index actual: **CREATE INDEX** idx_cust **ON** Orders (customer_id);

- EXPLAIN arată Index Scan dar tot încarcă pagini din heap
- Cum putem elimina citirile din paginile heap ?
- Care sunt consecințele soluției propuse ?

Pentru a elimina citirile din paginile heap (heap fetch) trebuie sa refacem indexul in asa fel incat acesta sa contina din start si valorile coloanelor status si total, nu doar paginarea id-urilor.

Deci **CREATE INDEX idx_cust ON Orders INCLUDE(status, total)**

CONSECINTE: Elimina problema cu heap fetch, dar acest index ocupa mai mult spatiu deoarece include si statusul si totalul. De asemenea, la fiecare update/delete/insert sistemul trebuie sa actualizeze un index mult mai mare.

Exercițiu 4

- Tabela are 50 de milioane de înregistrări
- 99% din comenzi au status = 'completed'.
- 1% au status = 'pending'.
- Aplicația citește comenzile 'pending' de sute de ori pe secundă
- Un index normal pe coloana status poate fi ignorat de către optimizator. De ce? Dar dacă căutăm comenzile 'cancelled'.
- Propuneți un index mai bun. Verificați cu EXPLAIN ANALYZE.

Indexul normal de pe coloana status poate fi ignorat deoarece împartirea după coloana status va genera un heap în care paginarea se va face doar pe baza statusului. Fiind 50M înregistrări și cu 1% pending, va genera o pagină cu 500K înregistrări doar pentru pending și indexul va conține 50M de înregistrări.

Pentru eficientizare, se face un partial index cu

```
CREATE INDEX par_index ON Orders(customer_id)
```

```
Where status = 'Pending'
```

Totusi pe Cancelled se pastreaza indexul obisnuit

Avem următoarele tabele:

- Orders (Id, Customer_Id, Status, Created_at, Total, Email)
- Customers (Id, Name)
- Products (Id, Category_Id, Name, Price)
- OrderItems (Id, Order_Id, Product_Id, Quantity, Price, Value)

Și următoarea distribuție de operațiuni:

- 80% citiri: istoric comenzi per client, detalii comenzi istorice, căutare produse după categorie
 - 20% scrieri: comenzi noi inserate rapid, actualizări de status comenzi
- Propuneți o schemă (set de indecși) și specificați care indecși impactează performanța la scriere.

// istoric comenzi per client – query

Select c.id, C.Name, O.Created_at, O.Total

From Customers C

Inner join Orders O ON O.Customer_id = C.id

ORDER BY O.created_at desc;

1.From + join

2.Where

3.Group by, having

4.Select

5.ORDER BY

Rezolvarea mea

CREATE INDEX ind1 ON Orders(Customer_id, Created_at) INCLUDE (Total)

CREATE INDEX ind2 ON OrderItems(Order_id) INCLUDE (Product_id, Quantity)

CREATE INDEX ind3 ON Products(Category_Id) INCLUDE (Name, Price)

Rezolvare de pe seminar:

```
CREATE INDEX idx_orders_customer_id_created_total ON Orders (customer_id, created_at  
DESC, total);  
CREATE INDEX idx_order_items_order_id ON OrderItems (order_id);  
CREATE INDEX idx_order_items_product_id ON OrderItems (product_id);  
CREATE INDEX idx_products_cat_status_covering ON Products (category_id) INCLUDE (name,  
price);
```